TABLES & FORMS

DATA IN, DATA OUT



Show **data** in a grid — and **collect** it from your visitors.

Session 2 · still pure HTML — no CSS, no JavaScript

TODAY'S ROADMAP

1 · TABLES

Rows, cells and headers: `table` `tr` `th` `td`.

2 · TABLE PRO

Sections, captions and merged cells.

3 · FORMS

Let visitors type, pick and send.

4 · INPUT TYPES

Email, date, number, checkbox, radio, select…

5 · GOOD FORMS

Labels, validation, structure.

6 · QUIZ

Check what stuck.

7 · YOU CODE

Upgrade your resume.

8 · WRAP-UP

Recap and next steps.

SECTION 01

TABLES

Timetables, prices, scores, schedules — whenever data has rows and columns, you need a table.



WHY A TABLE?

A table shows **data in a grid**: each row is one record, each column is one kind of information.

Good uses:

- a class timetable
- a price list
- exam results
- a comparison of options

COURSE	DAY	ROOM
Web Dev	Monday	B12
Databases	Tuesday	A03

*Tables are for **data**, never for page layout. Layout is CSS's job (later!).*

YOUR FIRST TABLE

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Amal</td>
    <td>20</td>
  </tr>
  <tr>
    <td>Youssef</td>
    <td>22</td>
  </tr>
</table>
```

NAME	AGE
Amal	20
Youssef	22

The browser draws the grid. You only describe the data.

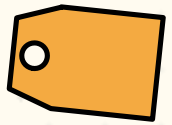
THE FOUR TAGS

`table`
The whole table. Everything lives inside it.

`tr`
Table **R**ow. One horizontal line of cells.

`th`
Table **H**eader cell. Bold, centered — a title.

`td`
Table **D**ata cell. A normal piece of data.



*Think in order: a **table** has **rows**, and a row has **cells**. You never write columns — they appear from the cells.*

ROWS FIRST, COLUMNS FOLLOW

Every `tr` must contain the **same number of cells** (unless you merge them — coming up).

```
<tr>
  <td>A1</td>
  <td>B1</td>
  <td>C1</td>
</tr>
<tr>
  <td>A2</td>
  <td>B2</td>
  <td>C2</td>
</tr>
```

A1	B1	C1
A2	B2	C2

3 cells per row → 3 columns.

TABLE PRO

Group rows by role, and give the table a title:

```
<table>
  <caption>Monthly budget</caption>
  <thead>
    <tr><th>Item</th><th>Cost</th></tr>
  </thead>
  <tbody>
    <tr><td>Rent</td><td>300</td></tr>
    <tr><td>Food</td><td>150</td></tr>
  </tbody>
  <tfoot>
    <tr><th>Total</th><td>450</td></tr>
  </tfoot>
</table>
```

Monthly budget

ITEM	COST
Rent	300
Food	150
TOTAL	450

caption = title · **thead** = header rows · **tbody** = main data · **tfoot** = totals. Screen readers love this.

MERGING CELLS

```html
<table>
  <tr>
    <th colspan="2">Monday</th>
  </tr>
  <tr>
    <td>8:30</td><td>Web Dev</td>
  </tr>
  <tr>
    <td rowspan="2">Lunch</td>
    <td>12:00</td>
  </tr>
  <tr>
    <td>13:00</td>
  </tr>
</table>
```

MONDAY	
8:30	Web Dev
Lunch	12:00
	13:00

`colspan="2"` → stretch across 2 **columns**.

`rowspan="2"` → stretch down 2 **rows**. The cells it covers are **not** written again.

TABLE CHEAT SHEET

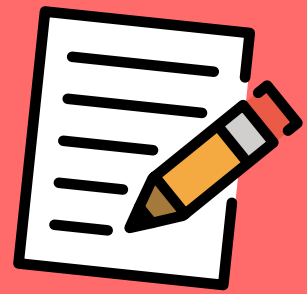
TAG / ATTRIBUTE	WHAT IT DOES	EXAMPLE
`table`	The whole table	`<table>…</table>`
`tr`	A row	`<tr>…</tr>`
`th` / `td`	Header cell / data cell	`<td>42</td>`
`caption`	Table title	`<caption>Budget</caption>`
`thead` `tbody` `tfoot`	Header / body / footer rows	`<tbody>…</tbody>`
`colspan` / `rowspan`	Merge cells across / down	`<td colspan="2">`

Table = rows of cells. Everything else is a refinement.

EXERCISE 1

BUILD AN INVOICE

Tables in the real world: structure a complete invoice, using everything from this section.



EXERCISE 1 — THE INVOICE

Create `invoice.html` with the **structure** of this invoice (no styling needed):

- a `caption` with the invoice number
- a `thead` with 4 columns: *Description*, *Qty*, *Unit price*, *Total*
- a `tbody` with **3 items**
- a `tfoot` with **Subtotal**, **VAT (20%)** and **Total**, where the label cell uses `colspan="3"`

Invoice #2026-014

DESCRIPTION	QTY	UNIT PRICE	TOTAL
Web hosting	1	50	50
Domain name	2	12	24
Support	3	20	60
SUBTOTAL			134
VAT (20%)			26.80
TOTAL			160.80

*Bonus: add a header row with **your company** and the **client** (use `colspan`).*

EXERCISE 1 — SOLUTION

```
<table>
  <caption>Invoice #2026-014</caption>
  <thead>
    <tr><th>Description</th><th>Qty</th>
        <th>Unit price</th><th>Total</th></tr>
  </thead>
  <tbody>
    <tr><td>Web hosting</td><td>1</td>
        <td>50</td><td>50</td></tr>
    <tr><td>Domain name</td><td>2</td>
        <td>12</td><td>24</td></tr>
    <tr><td>Support</td><td>3</td>
        <td>20</td><td>60</td></tr>
  </tbody>
  <tfoot>
    <tr><th colspan="3">Subtotal</th><td>134</td></tr>
    <tr><th colspan="3">VAT (20%)</th><td>26.80</td></tr>
    <tr><th colspan="3">Total</th><td>160.80</td></tr>
  </tfoot>
</table>
```

CHECK YOURSELF

Every body row has **4 cells**. Every footer row is **3 + 1**: one cell stretched over 3 columns, plus the amount.

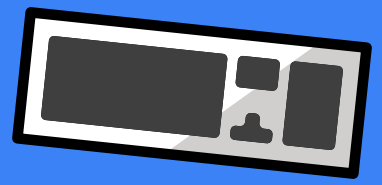
COMMON SLIP

Writing `colspan="3"` *and* 3 empty cells → the row gets too wide. Merged cells are **not** written again.

SECTION 02

FORMS

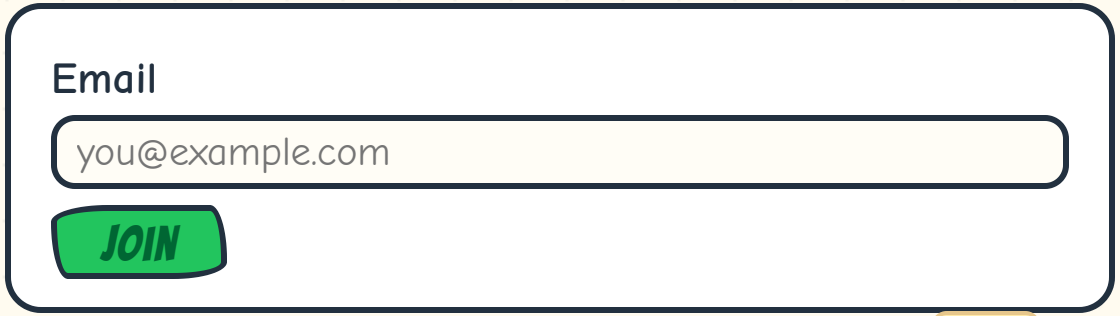
Login, sign-up, search, contact... every time you type something on a website, you're using a form.



Anatomy of a form

```
<form action="/send" method="post">
  <label for="email">Email</label>
  <input type="email" id="email"
         name="email">

  <button type="submit">Join</button>
</form>
```



action = where the data is sent · **method** = how (`get` or `post`).

Without a server to receive it, a form can't really "send" anything yet. Today we build the ***front*** *of the form.*

`label` + `input` = A PAIR

```html
<label for="city">City</label>
<input type="text" id="city" name="city">
```



`for` = `id`

The label's `for` must match the input's `id`. Then clicking the label focuses the field.

`name`

The key under which the value is sent. No `name` → the data is lost.

`type`

Decides what kind of field it is. Default: `text`.

INPUT TYPES (1/2) — TEXT-LIKE

`type`	GIVES YOU	BONUS
`text`	One line of text	the default
`email`	An email address	checks for `@`
`password`	Hidden characters	shows dots
`number`	A number	arrows, rejects letters
`date`	A calendar picker	no typing errors

EMAIL
you@example.com

PASSWORD
••••••••

NUMBER
3

INPUT TYPES (2/2) – CHOICES

```html
<!-- checkbox: 0, 1 or many -->
<input type="checkbox" id="h" name="skill">
<label for="h">HTML</label>

<!-- radio: one per name -->
<input type="radio" id="m" name="lvl">
<label for="m">Beginner</label>
<input type="radio" id="a" name="lvl">
<label for="a">Advanced</label>
```

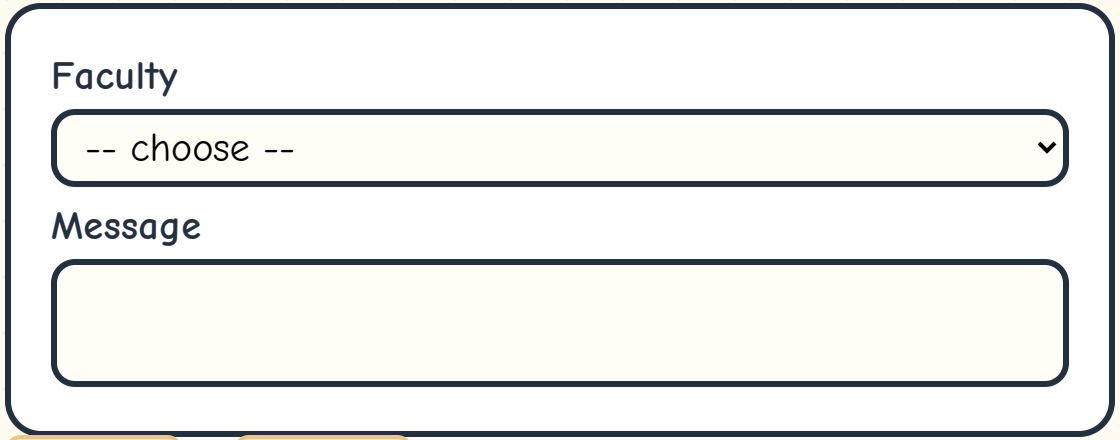


Radios with the **same** `name` form a group: pick only one.

DROPDOWN & TEXT AREA

```html
<label for="uni">Faculty</label>
<select id="uni" name="faculty">
  <option value="">-- choose --</option>
  <option value="sci">Sciences</option>
  <option value="law">Law</option>
</select>

<label for="msg">Message</label>
<textarea id="msg" name="msg"
          rows="4"></textarea>
```



`select` + `option` → one pick from a list.

`textarea` → several lines (it has a closing tag!).

BUTTONS

```
<button type="submit">Send</button>    <!-- sends the form -->
<button type="reset">Clear</button>    <!-- empties the fields -->
<button type="button">Click me</button>  <!-- does nothing yet -->
```

SUBMIT

The default inside a form. Triggers the sending.

RESET

Puts every field back to its start value.

BUTTON

No built-in action. JavaScript will use it later.

Always write the `type` *explicitly — a forgotten one defaults to* `submit`.

SECTION 03

GOOD FORMS

A form that works is nice. A form that is clear, safe to fill, and accessible is professional.



HELP THE USER — FREE VALIDATION

HTML can check input **before** it is sent. Just add attributes:

```
<input type="text" name="user"
       required
       minlength="3"
       placeholder="Your name">

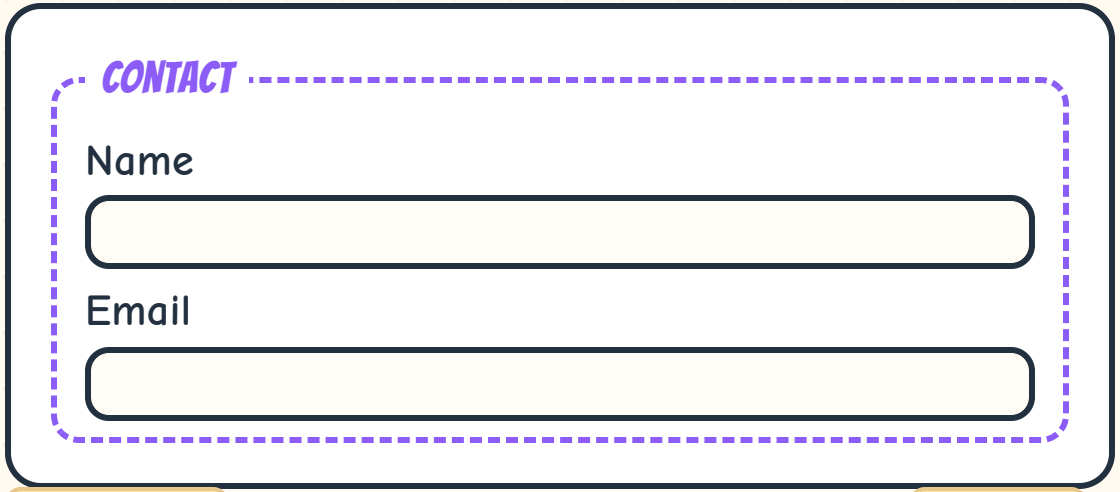
<input type="number" name="age"
       min="16" max="99">
```

- `required` → cannot be left empty
- `minlength` / `maxlength` → text length
- `min` / `max` → number or date range
- `placeholder` → grey hint inside the field
- `value` → pre-filled content

*A `placeholder` is a hint, **not** a replacement for a `label`.*

GROUP FIELDS

```
<fieldset>
  <legend>Contact</legend>
  <label for="n">Name</label>
  <input type="text" id="n" name="n">
  <label for="e">Email</label>
  <input type="email" id="e" name="e">
</fieldset>
```



`fieldset` draws a box around related fields; `legend` is its title.

FORM CHECKLIST

- Every field has a visible `label` linked with `for` / `id`
- Every field has a `name`
- Pick the **right** `type` (email, number, date…)
- Mark mandatory fields with `required`
- Group related fields with `fieldset`
- The button has an explicit `type`

Tick all six and your form is already better than half of the web.

A FULL EXAMPLE

```html
<form action="/register" method="post">
  <label for="n">Name</label>
  <input type="text" id="n" name="n" required>

  <label for="d">Birth date</label>
  <input type="date" id="d" name="d">

  <label for="f">Faculty</label>
  <select id="f" name="f">
    <option>Sciences</option>
    <option>Law</option>
  </select>

  <input type="checkbox" id="ok" name="ok">
  <label for="ok">I accept the rules</label>

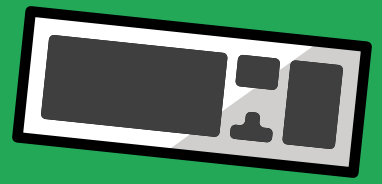
  <button type="submit">Register</button>
</form>
```



EXERCISE 2

BUILD A FORM

Put the whole forms toolbox together in one registration page.



EXERCISE 2 — EVENT REGISTRATION

Create `register.html` — a sign-up form for a student event:

1. A `fieldset` **"About you"**: full name (required), email (required), birth date
2. A `fieldset` **"Your event"**: a `select` of 3 workshops, **radio** buttons for the day (Saturday / Sunday), **checkboxes** for the meals you want
3. A `textarea` for questions
4. A required **checkbox** "I accept the rules"
5. **Submit** and **reset** buttons

RULES

Every field: a linked `label`, a `name`, the right `type`. Radios share one `name`.

TEST IT

Click each label. Submit empty — does the browser complain? Try a bad email.

EXERCISE 2 — SOLUTION

```html
<form action="#" method="post">
  <fieldset>
    <legend>About you</legend>
    <label for="n">Full name</label>
    <input type="text" id="n" name="name" required>
    <label for="e">Email</label>
    <input type="email" id="e" name="email" required>
    <label for="b">Birth date</label>
    <input type="date" id="b" name="birth">
  </fieldset>

  <!-- fieldset "Your event" → next column -->

  <label for="q">Questions</label>
  <textarea id="q" name="questions"></textarea>
  <input type="checkbox" id="ok" name="ok" required>
  <label for="ok">I accept the rules</label>

  <button type="submit">Register</button>
  <button type="reset">Clear</button>
</form>
```

```html
<fieldset>
  <legend>Your event</legend>
  <label for="w">Workshop</label>
  <select id="w" name="workshop">
    <option>HTML</option>
    <option>CSS</option>
    <option>Git</option>
  </select>
  <input type="radio" id="sat" name="day">
  <label for="sat">Saturday</label>
  <input type="radio" id="sun" name="day">
  <label for="sun">Sunday</label>
  <input type="checkbox" id="lu" name="meal">
  <label for="lu">Lunch</label>
</fieldset>
```

Check: both radios share `name="day"` 48

every field has a `for` / `id` pair.

SECTION 04

QUIZ TIME

Six questions. Answers right after. No pressure!



QUESTION 1

1. Which tag creates a table **row**? **A.** `<td>` **B.** `<tr>` **C.** `<th>`

2. How do you make a cell span **3 columns**? **A.** `rowspan="3"` **B.** `colspan="3"` **C.** `width="3"`

3. Should you use a table to position your page layout? **A.** Yes **B.** Only for headers **C.** No — tables are for data

QUESTION 2

4 What links a `label` to its `input`? **A.** `for` = `id` **B.** `name` = `type` **C.** `href` = `src`

5 How do you let users choose **exactly one** option among several? **A.** checkboxes **B.** radios sharing a `name` **C.** a textarea

6 Which attribute stops a form being sent with an empty field? **A.** `placeholder` **B.** `value` **C.** `required`

ANSWERS

1 B — `<tr>`. `td` and `th` are cells *inside* a row.

2 B — `colspan="3"`. `rowspan` merges downward instead.

3 C — **no.** Tables describe data; layout is done with CSS.

4 A — `for` = `id`. Clicking the label then focuses the field.

5 B — **radios with the same** `name`. Checkboxes allow several.

6 C — `required`. A placeholder is only a hint.

SECTION 05

YOUR TURN

Upgrade your resume with a table and a contact form. Still HTML only.



YOUR MISSION

Open your `resume.html` from last session and add:

1. **A skills table** — columns: *Skill*, *Level*, *Years*. Use `caption`, `thead` and `tbody`.
2. **A timetable or experience table** using at least one `colspan` or `rowspan`.
3. **A contact form** with: name, email, a subject `select`, a message `textarea`, and a submit button.

RULES

HTML only. Same file. Every field has a `label`, a `name`, and the right `type`. Mandatory fields use `required`.

TIP

Build the table first, refresh, then the form. Test by clicking labels and trying to submit empty.

STARTER SNIPPET

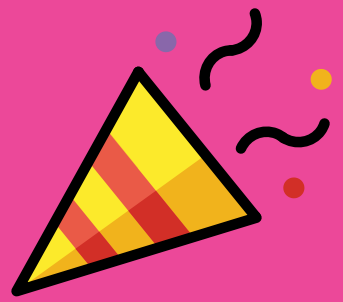
```
<h2>Skills</h2>
<table>
  <caption>My skills</caption>
  <thead>
    <tr><th>Skill</th><th>Level</th><th>Years</th></tr>
  </thead>
  <tbody>
    <tr><td>HTML</td><td>Beginner</td><td>0</td></tr>
  </tbody>
</table>

<h2>Contact me</h2>
<form action="#" method="post">
  <label for="name">Name</label>
  <input type="text" id="name" name="name" required>
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <label for="msg">Message</label>
  <textarea id="msg" name="msg"></textarea>
  <button type="submit">Send</button>
</form>
```

SECTION 06

RECAP

Two big skills in one session: displaying data and collecting it.



WHAT YOU LEARNED

TABLES

`table` → `tr` → `th` / `td`. Refine with `caption`, `thead`, `colspan`, `rowspan`.

FORMS

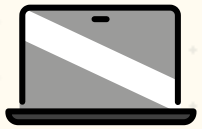
`form` wraps `label` + `input` / `select` / `textarea` and a `button`.

QUALITY

Link labels, name fields, pick the right `type`, add `required`.

*Next time: **CSS** — make your tables and forms beautiful.*

NOW LET'S CODE


Add a table and a form to `resume.html`.

Ask AI when you're stuck — then **understand every line**.

Questions? Hands up. Let's build.